# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

## CO1 ASSESSMENT TOOL 2 – Agile Sprint Planning & User Story Workshop

| | | | |
|---|---|---|---|
| **Course Code:** | CSA10 / CSA1063 | **Course Title:** | Software Engineering |
| **CO Assessed:** | CO1 | **Assessment:** | Assessment Tool 2 – **Agile Sprint Planning & User Story Workshop** |
| **Weightage:** | 20% | **Total Marks:** | 25 |
| **CO1 Statement:** | *Apply advanced methodologies and agile project management techniques to manage software projects effectively* | | |
| **Student Name:** | SWAMI SUJIT | **Reg. No.:** | 192525031 |
| **Date:** | 12/10/2026 | **Duration:** | 60 minutes |

**Code of Conduct**

*I _____SWAMI SUJIT_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____S.Sujit_____

## Annexure A – Agile Sprint Planning & User Story Workshop

**Instructions to Students:**

Each student is assigned an individual software project problem statement. Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

1. **Identify the Project Vision and Stakeholders** o Define the project's vision, objectives, and identify all key stakeholders involved.
2. **Create Epics and User Stories** o Break down the requirements into Epics and formulate well-structured User Stories using the format: *As a <user>, I want <feature>, so that <benefit>.*
3. **Define Acceptance Criteria** o Write clear and measurable acceptance criteria for each selected user story using the Given–When–Then format or equivalent.
4. **Prioritize the Product Backlog** o Organize and prioritize the product backlog based on business value, customer needs, and project dependencies.
5. **Plan the Sprint Backlog** o Select high-priority user stories for the first sprint and divide them into actionable development tasks.
6. **Estimate Effort Using Story Points** o Assign story points to each selected user story using an appropriate estimation technique (e.g., Fibonacci sequence or Planning Poker) and justify your estimates.
7. **Present the Sprint Goal and Expected Deliverables** o Define a clear sprint goal and describe the expected sprint deliverables, including the features to be completed at the end of the sprint.

# Intelligent Software License Compliance Monitoring Platform

**Industry Context**

**Organizations use hundreds of commercial and open-source software packages. Untracked installations can create licensing, security, and compliance risks.**

**Problem Statement**

**Design and develop a software-license management platform that tracks installations, licenses, renewal dates, usage, approved versions, and compliance status.**

**Objectives**

- **Maintain software inventory.**
- **Track license ownership.**
- **Monitor license expiry.**
- **Identify unauthorized installations.**
- **Track software usage.**
- **Generate compliance reports.**

**Expected Outcomes**

- **Improved license visibility.**
- **Reduced compliance issues.**
- **Better software budgeting.**
- **Reduced unauthorized usage.**
- **Automated license monitoring.**

# AGILE SPRINT PLANNING AND USER STORY
# Intelligent Software License Compliance Monitoring Platform

## Problem Statement

Organizations use hundreds of commercial and open-source software packages, and untracked installations create licensing, security and compliance risks. The task is to design and develop a software-license management platform that tracks installations, licenses, renewal dates, usage, approved versions and compliance status. In this document the platform is called LicenseGuard.

**Objectives:** maintain the software inventory; track license ownership; monitor license expiry; identify unauthorized installations; track software usage; and generate compliance reports.

**Expected outcomes:** improved license visibility, fewer compliance issues, better software budgeting, reduced unauthorized usage, and automated license monitoring.

Figure 1 shows the core idea of the platform: every installation found on a device is compared with the approved inventory and license records, and anything that fails the check is flagged and reported.

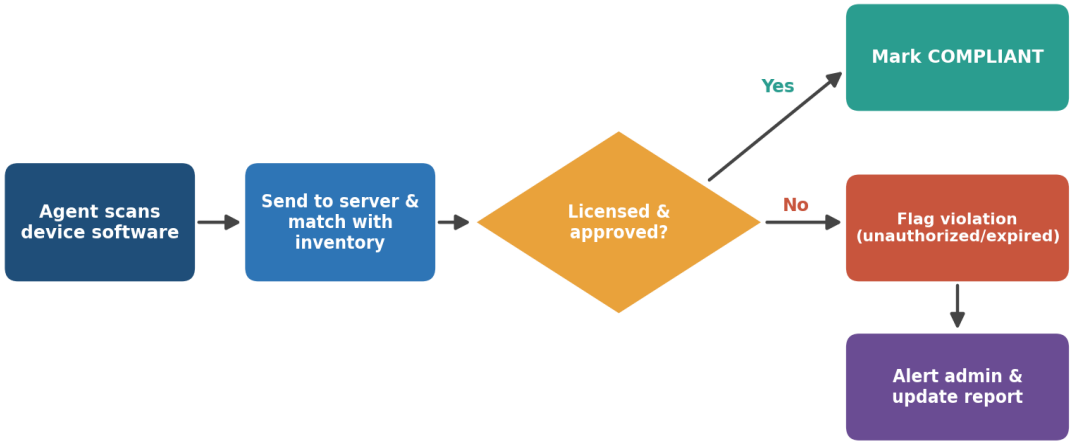



*Fig1: Compliance checking flow of LicenseGuard*

## Project Vision and Stakeholders

**Vision.** For IT and compliance teams who cannot see what software is installed or whether it is licensed, LicenseGuard is a license management platform that discovers installations, tracks licenses and renewals, and reports compliance automatically, unlike manual spreadsheets that are outdated and error prone.

Measurable goals: discover at least 95% of installed software automatically; send expiry alerts 60, 30 and 7 days before renewal; flag an unauthorized installation within 24 hours of detection; and generate a compliance report in under one minute.

The key stakeholders and their roles are as follows. The IT administrator maintains the inventory and approved versions and acts on violations. The license or procurement manager records purchases, owners and renewals. The compliance or audit officer reviews reports and prepares for vendor audits. The finance head uses cost and renewal data for software budgeting, and management (CIO) wants an overall compliance picture. Employees and department heads are the users whose devices are scanned and who request software. The security team cares about unpatched or unapproved versions. Software vendors issue licenses and may audit the organization. Inside the project team, the Product Owner sets priorities, the Scrum Master guides the process, and the developers and testers build and verify the product.

## Epics and User Stories

The requirements are grouped into six epics that follow the project objectives, and each epic is broken into user stories in the format As a <user>, I want <feature>, so that <benefit>. A common login story (US-10) supports all epics. Figure 2 shows the structure.

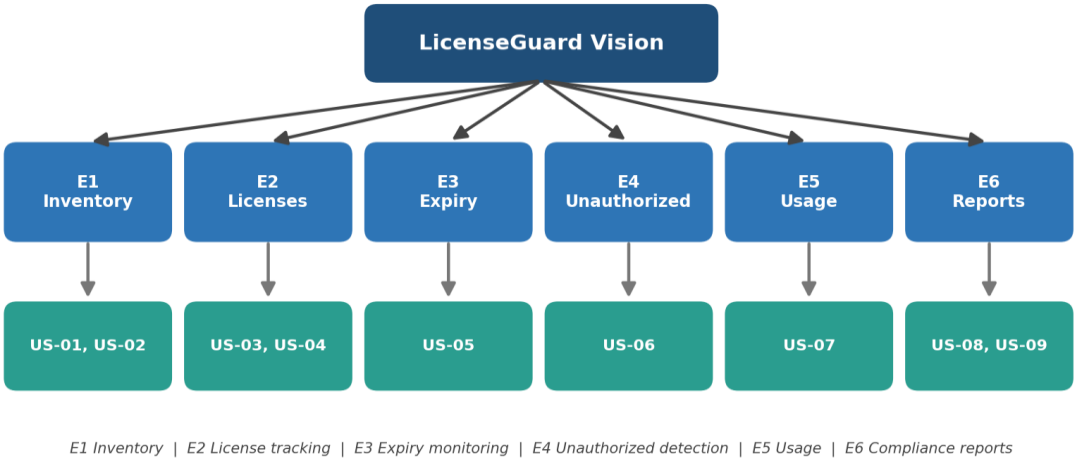



*Fig2: Vision, epics and user stories*

- **US-01:** As an IT admin, I want to register software with its approved versions, so that only authorized versions are treated as compliant.
- **US-02:** As an IT admin, I want an agent to scan devices and report installed software, so that the inventory is always up to date.
- **US-03:** As a license manager, I want to record a license with owner, key, seat count and expiry date, so that ownership and entitlement are clear.
- **US-04:** As a license manager, I want to assign licenses to users or departments, so that seat usage is traceable.
- **US-05:** As a license manager, I want automatic expiry alerts, so that licenses are renewed before they lapse.
- **US-06:** As a compliance officer, I want unapproved software or versions to be flagged, so that unauthorized usage is removed quickly.
- **US-07:** As a finance head, I want to see how often each software is used, so that unused licenses can be cancelled to save cost.
- **US-08:** As a compliance officer, I want to generate compliance reports, so that audits can be answered with evidence.
- **US-09:** As a manager, I want a dashboard of compliance status and renewal costs, so that I can plan the budget.
- **US-10:** As a user, I want to log in with a role-based account, so that only permitted people can view or change data.

## Acceptance Criteria

Criteria are written in Given–When–Then form for the stories selected for Sprint 1.

**US-10 Login.** Given a registered admin, when they enter valid credentials, then the dashboard opens within 3 seconds. Given wrong credentials, when login is attempted, then access is denied with an error message and the account locks after 5 failed attempts.

**US-01 Approved software.** Given an admin is on the Add Software page, when they enter a name, vendor and approved version list and save, then the record appears in the inventory. Given the same name and version already exist, when the admin saves again, then a duplicate error is shown.

**US-03 License record.** Given a license manager enters owner, key, seat count and expiry date, when they save, then the license is stored and shown as Active. Given an expiry date in the past, when saved, then the license is marked Expired.

**US-02 Device scan.** Given an agent is installed on a device, when a scan runs, then the list of installed software with versions reaches the server within 5 minutes. Given a device is offline, when it reconnects, then the pending scan is uploaded automatically.

## Prioritizing the Product Backlog

Stories were ranked by business value, customer need and dependencies. Login comes first because every feature needs secure access. The approved-software and license records come next because detection, alerts and reports all compare against them. Device scanning follows, and unauthorized-installation detection (US-06) can only be built after both the inventory and the scan exist, so it moves to Sprint 2 even though its business value is high. Reports and the dashboard depend on all collected data and therefore come last.

| Rank | ID | User story (short) | MoSCoW | Points | Planned sprint |
|---|---|---|---|---|---|
| 1 | US-10 | Role-based login | Must | 3 | Sprint 1 |
| 2 | US-01 | Register approved software | Must | 3 | Sprint 1 |
| 3 | US-03 | Record license details | Must | 5 | Sprint 1 |
| 4 | US-02 | Device scan agent | Must | 8 | Sprint 1 |
| 5 | US-06 | Detect unauthorized installs | Must | 8 | Sprint 2 |
| 6 | US-05 | License expiry alerts | Must | 5 | Sprint 2 |
| 7 | US-04 | Assign licenses | Should | 3 | Sprint 2 |
| 8 | US-08 | Compliance reports | Should | 5 | Sprint 3 |
| 9 | US-07 | Usage tracking | Should | 5 | Sprint 3 |
| 10 | US-09 | Budget dashboard | Could | 5 | Sprint 3 |

## Sprint Backlog and Task Breakdown

With a two-week sprint and an estimated capacity of about 20 points, the first four stories (19 points) are selected. Each is divided into tasks that one person can finish in a day or less.

| Story | Development tasks | Owner |
|---|---|---|
| US-10 Login | Design login screen; create users and roles table; build login API with password hashing; add lockout rule; test | Backend + Tester |
| US-01 Approved software | Design form; create software and version tables; build add/edit API; duplicate check; test | Full-stack dev |
| US-03 License record | Design license form; create license table with owner, seats, expiry; build save API; auto-status logic (Active/Expired); test | Full-stack dev |
| US-02 Device scan | Write agent to read installed programs; define upload API; store scan results; handle offline retry; test on 3 devices | Agent dev + Tester |

## Story Point Estimation

Estimation uses the Fibonacci sequence (1, 2, 3, 5, 8) with Planning Poker, where each member votes secretly and differences are discussed until the team agrees. The simplest story, Register approved software, is the 3-point reference: a basic form, one table and low risk. Login also gets 3 because it is a well-known pattern. License records get 5 as they have more fields, validation and status rules. The device scan gets 8, the highest value, because it needs an agent working across devices, offline handling and data matching, which brings the most uncertainty. The total is 3 + 3 + 5 + 8 = 19 points, which fits the sprint capacity.

## Sprint Goal and Expected Deliverables

**Sprint goal:** Allow authorized admins to log in, maintain approved software and license records, and automatically discover the software installed on devices.

At the end of the sprint the team will deliver a working role-based login module, an approved software and version register, a license record module with owner, seats and expiry status, and a scanning agent that uploads installed-software data to the central inventory. All of it will be tested against the acceptance criteria and shown in the sprint review. Detection of unauthorized installs, expiry alerts, usage tracking and reports stay in the backlog for later sprints. Progress will be tracked with a burndown chart; Figure 3 shows a sample for the 19 points.

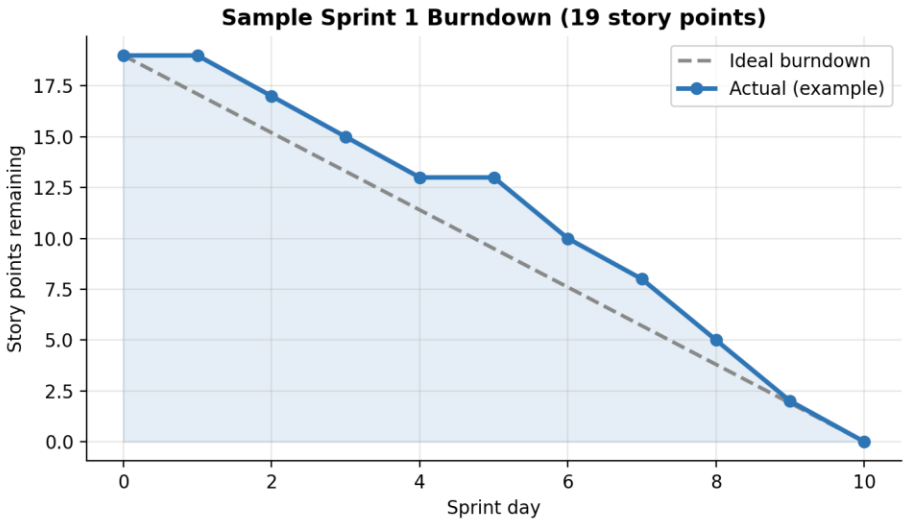



*Fig3: Sample Sprint 1 burndown chart*

## Rubrics for Calculation

| Criteria | Excellent (5 Marks) | Good (3 Marks) | Satisfactory (2 Marks) | Needs Improvement (1 Mark) |
|---|---|---|---|---|
| **Project Vision & Stakeholder Identification (30%)** | Project vision is clear, comprehensive, and aligned with the problem statement. All relevant stakeholders and their roles are correctly identified. | Vision is clear with minor omissions. Most stakeholders are correctly identified. | Vision is partially defined. Some stakeholders are missing or incorrectly identified. | Vision is unclear or incomplete. Stakeholders are poorly identified or missing. |
| **Epics & User Stories (25%)** | Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements. | Epics and user stories are mostly complete with minor formatting or requirement issues. | User stories are partially complete with limited Agile structure. | User stories are incomplete, unclear, or do not follow Agile practices. |
| **Acceptance Criteria (20%)** | Acceptance criteria are specific, measurable, testable, and aligned with all user stories. | Acceptance criteria are mostly clear and testable with minor gaps. | Acceptance criteria are partially defined and lack clarity or completeness. | Acceptance criteria are missing or not aligned with user stories. |
| **Product Backlog Prioritization & Sprint Planning (15%)** | Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete. | Backlog prioritization and sprint planning are mostly appropriate with minor issues. | Prioritization or sprint planning is partially complete with limited justification. | Backlog prioritization and sprint planning are poorly organized or incomplete. |

| | | | | |
|---|---|---|---|---|
| **Story Point Estimation, Sprint Goal & Presentation (10%)** | Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized. | Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear. | Estimation or sprint goal lacks justification. Presentation is adequate but incomplete. | Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete. |

| Criteria | **Mark** |
|---|---|
| Project Vision & Stakeholder Identification **(30%)** | /5 |
| Epics & User Stories **(25%)** | /5 |
| Acceptance Criteria **(20%)** | /5 |
| Product Backlog Prioritization & Sprint Planning **(15%)** | /5 |
| Story Point Estimation, Sprint Goal & Presentation **(10%)** | /5 |
| **TOTAL** | /25 |